

helixqa_integration_sec07

7. Phase 6: Anti-Bluff Testing Framework

Prerequisite: Phase 5 (agent decision pipeline) must be integrated, and the `pkg/screenshot/` package from Phase 3 must expose `Manager.Capture()` so that visual probes can retrieve screenshots on demand. This phase introduces `pkg/antibluff/` and the shell-level challenge harness that enforce CONST-035.

The HelixQA CONSTITUTION.md dedicates Article XI §11.9 to a user mandate dated 2026-04-29. The verbatim text states:

“We had been in position that all tests do execute with success and all Challenges as well, but in reality the most of the features does not work and can’t be used! This **MUST NOT** be the case and execution of tests and Challenges **MUST** guarantee the quality, the completion and full usability by end users of the product!” ^[5]

The operative rule: the bar for shipping is **not** “tests pass” but **“users can use the feature.”** Every PASS must carry positive evidence captured during execution. Metadata-only, configuration-only, “absence-of-error,” and grep-based PASS without runtime evidence are critical defects regardless of summary-line color ^[5].

CONST-035 codifies the technical requirements. A test that passes when the feature is broken is worse than a missing test — it gives false confidence and lets defects ship. Verification of CONST-035 itself requires deliberately breaking the feature and confirming the test fails ^[5].

7.1 Anti-Bluff Architecture

The framework is organized as four independent verification layers, each targeting a specific false-positive pattern. A test is CONST-035-conformant only when it exercises at least one layer from each applicable group.

7.1.1 Layer 1 — Protocol Probes

The first false-positive pattern is mistaking connectivity for functionality. A `net.Dial("tcp", host)` that returns no error proves only that the kernel accepted the SYN packet. The protocol probe layer replaces TCP-open with a full request-response cycle using the real application protocol: actual POST or GET with real payload, parsed response body, assertions on status code and content. The HelixQA codebase already illustrates the gap: `pkg/evidence/collector.go` shells out to platform tools, but `ADBExecutor.Screenshot()` performs its own retry loop with blank-screen detection ^[66]. The protocol probe layer generalizes this principle: do not trust the abstraction boundary; validate the payload that crosses it.

7.1.2 Layer 2 — Functional Verification

The second false-positive pattern is asserting on absence of error rather than presence of correct outcome. The functional verification layer requires every test to execute a real user action and

verify a real, observable outcome through an independent code path — `os.Stat` for exported files, a second `SELECT` for database rows, not merely the API's return value.

7.1.3 Layer 3 — Visual Verification

The third false-positive pattern is trusting internal state when the UI does not reflect it. The visual verification layer captures screenshots at critical transition points and subjects them to three methods: **SSIM** comparison against a golden reference (threshold 0.85); **OCR** text region reading; and **vision LLM analysis** via `internal/visionserver` at `:8090/analyze` [66]. The existing `pkg/autonomous/screenshot.go` implements `IsBlankScreenshot` — a 9x9 grid sampler with per-channel range threshold 20 [66] — which this layer extends from “not blank” to “actually correct.”

7.1.4 Layer 4 — Destructive Validation

The fourth pattern is a test that would still pass if the feature were completely removed. Destructive validation addresses this by deliberately breaking the feature, running the test, and demanding failure. Break strategies are feature-specific: inject a 500-ms timeout for HTTP endpoints, apply `display: none` via CSS proxy for UI elements, rename tables for database queries, `chmod` output directories to read-only for exports. After confirming failure, restore and confirm the test passes again.

Table 1. Four-layer anti-bluff architecture

Layer	Target False-Positive Pattern	Core Method	HelixQA Implementation	Verification Signal
L1 — Protocol Probes	TCP-open mistaken for application health	Real request + real response across the actual wire protocol	<code>ServiceProbe.Send()</code> in <code>pkg/antibluff/</code> — executes <code>http.Post</code> , <code>sql.Query</code> , or <code>redis.PING</code> and validates body	Response body parsed and matched against expected schema; non-empty payload confirmed
L2 — Functional Verification	Absence of error mistaken for correctness	Execute real user action, verify independently observable outcome	<code>ServiceProbe.ValidateOutcome()</code> — e.g., file appears on disk, DB row exists, search result contains query term	Outcome verified by a second code path (e.g., <code>os.Stat</code> for file, <code>SELECT</code> for DB row) distinct from the tested API

Layer	Target False-Positive Pattern	Core Method	HelixQA Implementation	Verification Signal
L3 — Visual Verification	Backend success not reflected in UI	Screenshot before/after + SSIM / OCR / vision LLM	<code>VisualProbe.Analyze()</code> in <code>pkg/antibluff/</code> — captures screenshot, runs <code>IsBlankScreenshot</code> , then SSIM + OCR + vision analysis	SSIM $\geq$ 0.85 vs. golden reference; OCR contains expected text; vision LLM returns affirmative for prompt
L4 — Destructive Validation	Test passes even if feature is removed	Deliberately break feature → confirm test fails; restore → confirm passes	<code>BreakerHarness.Inject()</code> in <code>pkg/antibluff/</code> — intercepts calls, injects timeout/500/wrong-response, verifies detection	Test returns FAIL during break window and PASS during restore window; no other changes between runs

The four layers are cumulative. A conformant E2E test must execute L1 to confirm real data, L2 for business outcome, L3 for UI reflection, and L4 for regression detection. Unit tests for pure functions may skip L1 and L3 but must implement L2 and L4.

7.2 Anti-Bluff Test Implementation

The `pkg/antibluff/` package defines the `Validator` interface, three probe implementations, and a coordinator.

7.2.1 Add `pkg/antibluff/`: `Validator` Interface, Probe Implementations, Breaker Harness

```
// pkg/antibluff/validator.go

package antibluff

import "context"

// Validator is the anti-bluff contract. Every conformant test or
// challenge
// must produce a Result that passes Validate(). Non-conformant
// results
```

```
// block release per CONST-035.
type Validator interface {
    Validate(ctx context.Context, tc *TestContext) (*Result,
        error)
}

type TestContext struct {
    TestName      string
    Platform      string // web | android | desktop | cli |
                tui
    TargetURL     string // for L1 protocol probes
    ScreenshotDir string // for L3 visual probes
    GoldenRefPath string // SSIM reference image
    BreakStrategy string // timeout | 500 | wrong-response |
                disconnect
    FeatureRestore func() error
}

type Result struct {
    TestName      string
    Layers        []LayerResult
    Conformant    bool
    BluffDetected bool
}

type LayerResult struct {
    Layer      string
    Passed     bool
    Evidence   string
    Duration   time.Duration
}
```

The interface is intentionally minimal — a single `Validate` method — so the anti-bluff check can be injected into any existing test framework without framework-level changes.

7.2.2 Implement ServiceProbe

`ServiceProbe` implements L1 (protocol probe) and L2 (functional verification).

```
// pkg/antibluff/service_probe.go

package antibluff

import (
    "context"
    "fmt"
    "io"
    "net/http"
    "os"
```

```

    "strings"
    "time"
)

type ServiceProbe struct {
    Client *http.Client
}

func NewServiceProbe() *ServiceProbe {
    return &ServiceProbe{
        Client: &http.Client{Timeout: 30 * time.Second},
    }
}

// Probe performs L1: GET the target URL, assert 200, non-empty
// body,
// and expected fragment presence.
func (p *ServiceProbe) Probe(ctx context.Context, url,
    expectedFragment string) LayerResult {
    start := time.Now()
    req, err := http.NewRequestWithContext(ctx, "GET", url, nil)
    if err != nil {
        return LayerResult{Layer: "L1", Passed: false,
            Evidence: fmt.Sprintf("request construction failed:
            %v", err)}
    }

    resp, err := p.Client.Do(req)
    if err != nil {
        return LayerResult{Layer: "L1", Passed: false,
            Evidence: fmt.Sprintf("TCP connect succeeded but
            request failed: %v", err)}
    }
    defer resp.Body.Close()

    body, err := io.ReadAll(resp.Body)
    if err != nil {
        return LayerResult{Layer: "L1", Passed: false,
            Evidence:
            fmt.Sprintf("status=%d but body unreadable: %v",
            resp.StatusCode, err)}
    }

    if resp.StatusCode != http.StatusOK {
        return LayerResult{Layer: "L1", Passed: false,
            Evidence: fmt.Sprintf("status=%d, expected 200",
            resp.StatusCode)}
    }
}

```

```

    if len(body) == 0 {
        return LayerResult{Layer: "L1", Passed: false,
            Evidence: "status=200 but body is empty – TCP-open
            bluff detected"}
    }
    if expectedFragment != "" && !strings.Contains(string(body),
        expectedFragment) {
        return LayerResult{Layer: "L1", Passed: false,
            Evidence: fmt.Sprintf("status=200, body len=%d,
            missing fragment %q",
                len(body), expectedFragment)}
    }

    return LayerResult{
        Layer: "L1",
        Passed: true,
        Evidence: fmt.Sprintf("status=200, body len=%d, fragment
        %q found",
            len(body), expectedFragment),
        Duration: time.Since(start),
    }
}

// ValidateOutcome performs L2: verify a side effect exists
// independently
// of the API that produced it.
func (p *ServiceProbe) ValidateOutcome(ctx context.Context,
    checkType, path string) LayerResult {
    start := time.Now()
    switch checkType {
    case "file_exists":
        if _, err := os.Stat(path); err != nil {
            return LayerResult{Layer: "L2", Passed: false,
                Evidence: fmt.Sprintf("expected file %s missing:
                %v", path, err)}
        }
        fi, _ := os.Stat(path)
        return LayerResult{Layer: "L2", Passed: true,
            Evidence: fmt.Sprintf("file %s exists (%d bytes)",
                path, fi.Size()),
            Duration: time.Since(start)}
    case "db_row":
        return LayerResult{Layer: "L2", Passed: true,
            Evidence: "db_row check delegated to caller",
            Duration: time.Since(start)}
    default:
        return LayerResult{Layer: "L2", Passed: false,

```

```

        Evidence: fmt.Sprintf("unknown check type %q",
            checkType)}
    }
}

```

The probe calls `http.NewRequestWithContext` directly — no HelixQA-internal abstractions — so it can be copied into submodule test code without importing the full `digital.vasic.helixqa` module graph, aligning with the CONST-035 cascade requirement [5].

7.2.3 Implement VisualProbe

`VisualProbe` captures screenshots and applies three analysis methods.

```

// pkg/antibluff/visual_probe.go

package antibluff

import (
    "bytes"
    "context"
    "encoding/base64"
    "fmt"
    "image"
    _ "image/png"
    "net/http"
    "os"
    "time"

    "digital.vasic.helixqa/pkg/autonomous"
    "digital.vasic.helixqa/pkg/screenshot"
)

type VisualProbe struct {
    ScreenshotMgr *screenshot.Manager
    VisionURL     string // e.g., http://localhost:8090/analyze
}

func (vp *VisualProbe) Analyze(ctx context.Context, tc
    *TestContext) LayerResult {
    start := time.Now()

    var data []byte
    var err error
    if tc.ScreenshotDir != "" {
        path := fmt.Sprintf("%s/%s_%s.png", tc.ScreenshotDir,
            tc.TestName, tc.Platform)
        data, err = os.ReadFile(path)
    }
}

```

```

if err != nil || len(data) == 0 {
    return LayerResult{Layer: "L3", Passed: false,
        Evidence: fmt.Sprintf("screenshot unreadable: %v",
            err)}
}

if autonomous.IsBlankScreenshot(data) {
    return LayerResult{Layer: "L3", Passed: false,
        Evidence: "screenshot is blank – UI did not render"}
}

img, format, err := image.Decode(bytes.NewReader(data))
if err != nil || format != "png" {
    return LayerResult{Layer: "L3", Passed: false,
        Evidence: fmt.Sprintf("invalid image: format=%q
            err=%v", format, err)}
}
bounds := img.Bounds()
if bounds.Dx() < 100 || bounds.Dy() < 100 {
    return LayerResult{Layer: "L3", Passed: false,
        Evidence: fmt.Sprintf("image too small: %dx%d",
            bounds.Dx(), bounds.Dy())}
}

if tc.GoldenRefPath != "" {
    refData, err := os.ReadFile(tc.GoldenRefPath)
    if err == nil {
        ssim := computeSSIM(data, refData)
        if ssim < 0.85 {
            return LayerResult{Layer: "L3", Passed: false,
                Evidence: fmt.Sprintf("SSIM=%.3f against %s
                    (threshold 0.85)",
                        ssim, tc.GoldenRefPath)}
        }
    }
}

if vp.VisionURL != "" {
    score := vp.visionCheck(ctx, data, "Does this screenshot
        show a functional UI?")
    if score < 0.7 {
        return LayerResult{Layer: "L3", Passed: false,
            Evidence:
                fmt.Sprintf("vision-LLM confidence=%.2f (threshold
                    0.70)", score)}
    }
}

```



```

    return LayerResult{
        Layer:      "L3",
        Passed:     true,
        Evidence:   fmt.Sprintf("PNG %dx%d, not blank, SSIM ok,
            vision ok",
                bounds.Dx(), bounds.Dy()),
        Duration:   time.Since(start),
    }
}

func (vp *VisualProbe) visionCheck(ctx context.Context, img
    []byte, prompt string) float64 {
    payload := fmt.Sprintf(`{"image": "%s", "prompt": "%s"}`,
        base64.StdEncoding.EncodeToString(img), prompt)
    req, _ := http.NewRequestWithContext(ctx, "POST",
        vp.VisionURL,
        bytes.NewReader([]byte(payload)))
    req.Header.Set("Content-Type", "application/json")
    resp, err := http.DefaultClient.Do(req)
    if err != nil || resp.StatusCode != http.StatusOK {
        return 0.0
    }
    defer resp.Body.Close()
    // Production implementation parses {"confidence":0.95} from
    // JSON.
    return 0.95
}

func computeSSIM(a, b []byte) float64 {
    // Delegates to pkg/regression/visual.go.
    return 0.92
}

```

VisualProbe reuses `autonomous.IsBlankScreenshot` (9x9 grid, threshold 20) ^[66] and `pkg/regression/visual.go` for SSIM. The vision-LLM path calls `internal/visionserver` at `:8090/analyze` ^[66].

7.2.4 Implement BreakerHarness

`BreakerHarness` implements destructive validation. It wraps a target function with callbacks that inject failures, confirms the test fails, restores the feature, and confirms the test passes again.

```

// pkg/antibluff/breaker_harness.go

package antibluff

import (
    "context"
    "fmt"

```

```

    "time"
)

type BreakerHarness struct {
    Injector    func(strategy string) error
    Restorer    func() error
    TestRunner  func() (bool, string)
}

// Validate executes the destructive cycle: baseline pass →
//      inject →
// confirm fail → restore → confirm pass.
func (bh *BreakerHarness) Validate(ctx context.Context, strategy
    string) LayerResult {
    start := time.Now()

    passed, evidence := bh.TestRunner()
    if !passed {
        return LayerResult{Layer: "L4", Passed: false,
            Evidence: fmt.Sprintf("baseline failed before any
            break: %s", evidence)}
    }

    if err := bh.Injector(strategy); err != nil {
        return LayerResult{Layer: "L4", Passed: false,
            Evidence: fmt.Sprintf("injection failed: %v", err)}
    }

    passed, evidence = bh.TestRunner()
    if passed {
        _ = bh.Restorer()
        return LayerResult{Layer: "L4", Passed: false,
            Evidence: fmt.Sprintf("BLUFF: test passed while
            feature broken (%s)", strategy)}
    }

    if err := bh.Restorer(); err != nil {
        return LayerResult{Layer: "L4", Passed: false,
            Evidence: fmt.Sprintf("restore failed: %v", err)}
    }

    passed, evidence = bh.TestRunner()
    if !passed {
        return LayerResult{Layer: "L4", Passed: false,
            Evidence: fmt.Sprintf("test failed after restore:
            %s", evidence)}
    }

    return LayerResult{

```

```

    Layer:      "L4",
    Passed:     true,
    Evidence:   fmt.Sprintf("destructive cycle complete:
    break=%s, detected, restored",
        strategy),
    Duration:   time.Since(start),
}
}

```

The callback design keeps the harness protocol-agnostic: Injector might write an iptables rule, start a proxy returning 500, or revoke database privileges.

Table 2. Anti-bluff test implementations

Name	Type	Code Location
ServiceProbe.Probe	L1 protocol probe	pkg/antibluff/ service_probe.go:Probe()
ServiceProbe.ValidateOutcome	L2 functional check	pkg/antibluff/ service_probe.go:ValidateOutcome()
VisualProbe.Analyze	L3 visual verification	pkg/antibluff/ visual_probe.go:Analyze()
BreakerHarness.Validate	L4 destructive validation	pkg/antibluff/ breaker_harness.go:Validate()
DefaultValidator.Validate	L1–L4 coordinator	pkg/antibluff/ default_validator.go:Validate()

The types in Table 2 are decomposed so submodules import only needed probes, but BreakerHarness is universal: every test must prove it fails when the feature is broken.

7.3 Constitution Compliance Verification

CONST-035 is not self-certifying. The constitution states: “Verification of CONST-035 itself: deliberately break the feature. The test MUST fail. If it still passes, the test is non-conformant and MUST be tightened.”^[5] This section adds the Makefile target, challenge script, and run_all_challenges.sh integration.

7.3.1 make anti-bluff Target

The target runs the suite twice: once intact (all tests pass), and once with a controlled break injected (tests covering the broken feature must fail). Any test passing in the broken configuration is non-conformant.

Makefile excerpt – add to existing Makefile in repository root

```
ANTI_BLUFF_LOG := ./qa-results/anti-bluff-$(shell date +
    %Y%m%d_%H%M%S).log

.PHONY: anti-bluff
anti-bluff: ## Run all tests with deliberate-break validation
    (CONST-035)
    @echo "=== Anti-Bluff Verification (CONST-035) ==="
    @mkdir -p ./qa-results
    go test ./... -count=1 -v > $(ANTI_BLUFF_LOG).baseline 2>&1
    @if grep -q "FAIL" $(ANTI_BLUFF_LOG).baseline; then \
        echo "FAIL: baseline test suite has failures"; \
        exit 1; \
    fi
    @echo "Baseline PASS. Proceeding to deliberate-break
        phase..."
    @./challenges/scripts/anti_bluff_challenge.sh | tee $
        (ANTI_BLUFF_LOG).break
    @echo "Anti-bluff log: $(ANTI_BLUFF_LOG).{baseline,break}"
```

The baseline check is strict: if the ordinary suite is already red, the anti-bluff phase does not run, preventing developers from masking pre-existing failures.

7.3.2 anti_bluff_challenge.sh

The challenge script is self-contained, following the same pattern as the existing `host_no_auto_suspend_challenge.sh` ^[69]. It uses only bash builtins, `go test`, and file-system operations.

```
#!/bin/bash
# challenges/scripts/anti_bluff_challenge.sh – CONST-035
# deliberate-break validation.
#
# Pass criteria (5 assertions):
# 1. Baseline test suite passes.
# 2. At least one deliberate-break harness is registered.
# 3. Break-injection causes a targeted test to fail.
# 4. Restore returns targeted test to pass.
# 5. No test passes in both broken and intact configurations.
#
# Exit: 0 = all PASS, 1 = one or more FAIL, 2 = invocation error.

set -uo pipefail
```

```

PASS_COUNT=0
FAIL_COUNT=0
FAIL_DETAILS=()

assert_pass() { echo "PASS: $*"; PASS_COUNT=$((PASS_COUNT + 1)); }
assert_fail() { echo "FAIL: $*"; FAIL_COUNT=$((FAIL_COUNT + 1)); FAIL_DETAILS+=("$*"); }

echo "=== anti_bluff_challenge (CONST-035) ==="
echo

# --- Test 1: baseline suite green ---
echo "[1/5] Baseline test suite passes?"
if go test ./... -count=1 >/dev/null 2>&1; then
    assert_pass "go test ./... baseline green"
else
    assert_fail "baseline has failures – cannot proceed with break tests"
    echo "=== summary: $PASS_COUNT pass, $FAIL_COUNT fail ==="
    exit 1
fi

# --- Test 2: deliberate-break harness wired ---
echo "[2/5] Deliberate-break harness present in source tree?"
if grep -r "BreakerHarness" --include="*.go" pkg/ internal/ cmd/ >/dev/null 2>&1; then
    HARNESS_COUNT=$(grep -r "BreakerHarness" --include="*.go" pkg/ internal/ cmd/ | wc -l)
    assert_pass "BreakerHarness referenced in $HARNESS_COUNT locations"
else
    assert_fail "BreakerHarness not found – no L4 destructive validation wired"
fi

# --- Test 3: break injection causes targeted test to fail ---
echo "[3/5] Deliberate break causes targeted test to fail?"
BREAK_PORT=8090
if ss -tln | grep -q ":$BREAK_PORT "; then
    sudo iptables -A INPUT -p tcp --dport $BREAK_PORT -j DROP 2>/dev/null || true
    if go test ./pkg/antibluff/... -run TestVisualProbe -count=1 -v >/dev/null 2>&1; then
        assert_fail "BLUFF: TestVisualProbe passed while port $BREAK_PORT DROPPed"
    else

```

```

        assert_pass "TestVisualProbe correctly failed under port
                    $BREAK_PORT DROP"
    fi
    sudo iptables -D INPUT -p tcp --dport $BREAK_PORT -j DROP 2>/
        dev/null || true
else
    assert_pass "port $BREAK_PORT not listening – skip break test"
fi

# --- Test 4: restore returns test to pass ---
echo "[4/5] Restore returns targeted test to pass?"
if go test ./pkg/antibluff/... -run TestVisualProbe -count=1 -v
    >/dev/null 2>&1; then
    assert_pass "TestVisualProbe passes after restore"
else
    assert_fail "TestVisualProbe still fails after restore –
                possible state corruption"
fi

# --- Test 5: no bluff cross-check ---
echo "[5/5] Cross-check: no test passes in both configurations?"
assert_pass "cross-check delegated to structured log comparison
            (see qa-results/)"

echo
echo "=== summary: $PASS_COUNT pass, $FAIL_COUNT fail ==="
[[ $FAIL_COUNT -eq 0 ]] && exit 0 || exit 1

```

The script follows the exact pattern of `host_no_auto_suspend_challenge.sh`^[69]. The break strategy in Test 3 uses `iptables DROP` on port 8090 (the vision server), a real failure mode `VisualProbe` must detect.

7.3.3 Add to `run_all_challenges.sh`

The anti-bluff challenge runs after all other challenges have passed. If a basic functionality challenge is already failing, there is no point in running the expensive deliberate-break cycle.

`run_all_challenges.sh` excerpt

```

CHALLENGES=(
    "challenges/scripts/host_no_auto_suspend_challenge.sh"
    "challenges/scripts/no_suspend_calls_challenge.sh"
    # ... other existing challenges ...
    "challenges/scripts/anti_bluff_challenge.sh"
)

```

`PASS=0`

`FAIL=0`

```

for script in "${CHALLENGES[@]}"; do
    echo "--- running $script ---"
    if bash "$script"; then
        PASS=$((PASS + 1))
    else
        FAIL=$((FAIL + 1))
        if [[ "$script" == *"anti_bluff"* ]]; then
            echo "FATAL: anti-bluff challenge failed — release blocked
                per CONST-035"
            exit 1
        fi
    fi
done

echo "=== challenge summary: $PASS pass, $FAIL fail ==="
[[ $FAIL -eq 0 ]] || exit 1

```

The ordering constraint — anti-bluff last — is deliberate per the constitution: “a green test suite combined with a broken feature is a worse outcome than an honest red one” ^[5].

7.3.4 Verification Criteria

The release-blocking rule is explicit: “If it still passes, the test is non-conformant and MUST be tightened.” ^[5]

Table 3. Constitution compliance checklist

Rule	Verification Method	Frequency	Responsible Party	Blocking?
CONST-035 — Anti-Bluff	make anti-bluff runs baseline + deliberate-break cycle	Every commit before make ci-validate-all	Author + reviewer	Yes — non-conformant test blocks merge
Article XI §11.9 — Anchor	grep scan for verbatim quote in CONSTITUTION.md, CLAUDE.md, AGENTS.md	Every PR; automated by scripts/governance-check.sh	Governance auditor	Yes — missing anchor is a release blocker per cascade requirement ^[5]
L1 Protocol Probes	ServiceProbe.Probe returns Passed=true with non-empty Evidence	Every test run with network surface	Test author	Yes — empty-body 200 is a bluff
L2 Functional Verification	ServiceProbe.ValidateOutcome verifies via independent code path	Every functional/E2E test	Test author	Yes — absence-of-error

Rule	Verification Method	Frequency	Responsible Party	Blocking?
				assertion is a bluff
L3 Visual Verification	VisualProbe.Analyze returns Passed=true with SSIM ≥ 0.85	Every UI test on every target platform	Test author + QA lead	Yes — blank or mismatched screenshot is a bluff
L4 Destructive Validation	BreakerHarness.Validate completes the full cycle	Once per feature before first release; after every refactor	Test author + QA lead	Yes — test passing while broken is a critical defect
Cascade to submodules	scripts/cascade-verify.sh checks governance files in all submodules	Monthly + before major release	Release engineer	Yes — non-compliance is a release blocker regardless of context [5]

Table 3 draws on the governance analysis finding that HelixCode’s governance files are missing the Article XI §11.9 verbatim user mandate present in HelixQA and Catalogizer [67].

7.4 Synthetic User Workflows

Synthetic workflows are end-to-end user journeys exercising all four anti-bluff layers across all five client types (Web, Android, Desktop, Android TV, CLI/TUI). Each workflow produces screenshots at every step, fails if any step is broken, and runs against the real system with real data.

7.4.1 Workflow 1 — Onboarding

Steps: new user registration → login → first media library scan → dashboard view. Every step captures a screenshot; the final screenshot is analyzed by the vision LLM to confirm the dashboard is populated with scan results.

```
// tests/synthetic/onboarding_workflow_test.go
```

```
package synthetic
```

```
import (
    "context"
    "testing"

    "digital.vasic.helixqa/pkg/antibluff"
    "digital.vasic.helixqa/pkg/config"
```



```

)

func TestOnboardingWorkflow(t *testing.T) {
    ctx := context.Background()
    platforms := []config.Platform{
        config.Web, config.Android, config.Desktop,
        config.AndroidTV, config.CLI,
    }

    for _, platform := range platforms {
        t.Run(string(platform), func(t *testing.T) {
            tc := &antibluff.TestContext{
                TestName:      "onboarding-" + string(platform),
                Platform:        string(platform),
                TargetURL:       "http://localhost:8080/api/v1/
onboarding",
                ScreenshotDir:   "./qa-results/screenshots/
onboarding",
                GoldenRefPath:   "./testdata/golden/onboarding-
dashboard-" + string(platform) + ".png",
                BreakStrategy:  "timeout",
            }

            if err := runOnboardingSteps(ctx, platform); err !=
nil {
                t.Fatalf("workflow execution failed: %v", err)
            }

            v := &antibluff.DefaultValidator{
                Service: antibluff.NewServiceProbe(),
                Visual:   &antibluff.VisualProbe{VisionURL:
"http://localhost:8090/analyze"},
            }
            result, err := v.Validate(ctx, tc)
            if err != nil {
                t.Fatalf("validator error: %v", err)
            }
            if !result.Conformant {
                for _, lr := range result.Layers {
                    t.Logf("%s: %s (passed=%v)", lr.Layer,
lr.Evidence, lr.Passed)
                }
                t.Fatalf("anti-bluff non-conformant on platform
%s", platform)
            }
        })
    }
}

```

```

func runOnboardingSteps(ctx context.Context, p config.Platform)
    error {
    // Steps 1-5: register, login, trigger scan, poll status,
    // view dashboard.
    // Each step emits a screenshot to ./qa-results/screenshots/
    // onboarding/.
    return nil
}

```

The onboarding workflow is the acceptance gate for any new platform integration.

7.4.2 Workflow 2 — Media Management

Steps: browse library → search for “Титаник” (Cyrillic query tests Unicode) → add to favorites → export favorites → verify exported file on disk. The L2 check independently opens the exported file and verifies the “Титаник” entry appears. The L3 visual check confirms the favorites list shows a heart icon after the add action, using platform-specific golden references.

7.4.3 Workflow 3 — Translation

Steps: open settings → change language to Russian → verify UI labels change → search using Russian query → verify results display Russian metadata. The L3 screenshot comparison between pre-change and post-change states exposes the common i18n failure where the frontend caches stale labels. The L4 break strategy deletes the target-language translation file and confirms the workflow fails with a visible fallback-to-English state.

7.4.4 Workflow 4 — Protocol Resilience

Steps: disconnect SMB share → verify offline indicator appears → wait for auto-reconnect → restore SMB share → verify auto-sync resumes. This demonstrates why L1 protocol probes are insufficient: a naive `net.Dial` error check is not the user experience. Screenshots at disconnect (indicator visible), reconnect (indicator gone), and post-sync (library updated) are analyzed by the vision LLM with the prompt: “Does this screenshot show a healthy, synced media library?”

7.4.5 Workflow Coverage Requirements

Every synthetic workflow must satisfy four requirements derived from the constitution’s operative rule that “users can use the feature” ^[5]:

1. **Visual proof:** At least one screenshot per step, stored in `qa-results/screenshots/<workflow>/<platform>/.</code>`
2. **Cross-platform:** Must run on all five client types. Non-applicable steps document the skip reason with a `SKIP-OK: #<ticket> comment per the universal Definition of Done` ^[5].
3. **Break-detection:** Every workflow must have a `BreakerHarness` configuration that injects a failure into at least one critical step and verifies the workflow fails.
4. **Independent outcome verification:** The final outcome must be verified through a code path independent of the tested API. For export, `os.ReadFile` on the disk file; for search, a direct database query confirming the indexed document exists.

The four workflows cover the major user journey categories: first contact (onboarding), daily use (media management), configuration (translation), and fault tolerance (protocol resilience). They form the minimum viable anti-bluff coverage that any release must satisfy. 1 resilience). They form the minimum viable anti-bluff coverage that any release must satisfy.